

Práctica 1: Técnicas de Búsqueda de Poblaciones para el Problema de Portfolio

Andrés Borja

Curso académico 2025-2026

Índice

1. Descripción del problema	3
2. Aplicación de los algoritmos	4
2.1. Representación de las Soluciones	4
2.2. Retorno	4
2.3. Fórmulas	4
2.4. Conjuntos de datos	4
2.5. Clase Portfolio	4
2.6. Representación de la estructura Portfolio	4
2.7. Función Objetivo	5
2.8. Operadores Comunes	6
2.8.1. Solución válida	6
2.8.2. Generación de soluciones	6
2.8.3. Reparación de soluciones	6
2.8.4. Otros métodos	7
3. Descripción de los algoritmos	8
3.1. Algoritmo Greedy	8
3.2. Búsqueda Local (BL)	8
4. Estructura del código y manual de usuario	10
4.1. Estructura del código	10
4.2. Compilación	10
4.3. Ejecución	10
4.4. Gráficas	10
5. Experimentos y análisis de resultados	11
5.1. Descripción de los experimentos	11
5.1.1. Algoritmos comparados	11
5.2. Resultados	11
5.3. Análisis de resultados	13
5.3.1. Aleatorio	13
5.3.2. Búsqueda Local	13
5.3.3. Greedy	13
6. Referencias	14

1. Descripción del problema

El problema del portfolio consiste en encontrar la asignación óptima de capital entre un conjunto de N activos financieros disponibles, de forma que se maximice el beneficio y se minimice el riesgo.

Cada solución del problema se representa mediante un vector de pesos $w = (w_1, w_2, \dots, w_N)$, donde w_i indica la proporción de capital invertida en el activo i . Deberán de cumplir ciertas restricciones que ahora veremos.

El beneficio asociado a una solución se calcula a partir de los retornos históricos de los activos, mientras que el riesgo se modela mediante la matriz de covarianzas, que recoge la dependencia entre los distintos activos. De esta forma, el problema plantea un compromiso entre maximizar la rentabilidad esperada y minimizar la variabilidad de dicha rentabilidad.

Este problema lo vamos a abordar mediante el uso de técnicas metaheurísticas como los algoritmos greedy o la búsqueda local.

2. Aplicación de los algoritmos

2.1. Representación de las Soluciones

Para representar una solución se usará un vector de pesos reales $w = (w_1, w_2, \dots, w_n)$ donde cada elemento w_i representa la proporción del capital total que se invertirá en la empresa i . Las restricciones para que una solución sea válida son las siguientes:

- **Suma de pesos igual a 1:** $\sum_{i=1}^n w_i \approx 1$. Se pone $\approx$ porque con valores reales las comparaciones se hacen con un cierto margen de error, como 10^{-8} .
- **Rango limitado:** $(w_i == 0) \vee (lo \leq w_i \leq hi), \forall i \in 1..N$. Los valores lo y hi se definirán más adelante según los datos con los que trabajemos.

2.2. Retorno

Para cada día t del periodo y en cada empresa i tenemos el retorno obtenido μ_{ti} que viene dado por:

$$\mu_{ti} = \frac{V_{ti} - V_{(t+1)i}}{V_{ti}}$$

Un valor positivo indica que ha habido mejora, y un valor negativo indica que se ha perdido dinero ese día.

2.3. Fórmulas

A partir de la matriz de retornos podremos calcular:

Concepto	Para qué se usa	Fórmula
Media por acción	Retorno medio de una empresa en el periodo T .	$\bar{\mu}_i = (\sum_{t=1}^T \mu_{ti}) / T$
Covarianza	Medir cómo varían conjuntamente dos acciones para evaluar su dependencia.	$\Sigma_{ij} = \frac{1}{T} \sum_{t=1}^T (\mu_{ti} - \bar{\mu}_i) \cdot (\mu_{tj} - \bar{\mu}_j)$
Beneficio	Calcular la ganancia total esperada de la cartera usando logaritmos.	$Beneficio(w) = \sum_i w_i \cdot (\sum_{t=1}^T \log(1 + \mu_{ti}))$
Riesgo	Cuantificar la volatilidad o peligro de pérdida total de la cartera.	$Riesgo(w) = \sum_i \sum_j w_i \cdot \Sigma_{ij} \cdot w_j$

Cuadro 1: Métricas

2.4. Conjuntos de datos

Se utilizan datos bursátiles reales (2015-2025) de empresas con presencia continua. Los archivos CSV incluyen la fecha (**Date**) y la mejora relativa μ_{ti} por empresa:

- `ibex_35_data.csv`: IBEX 35 con 30 empresas y 2813 días ($lo = 0,005, hi = 0,08$).
- `syp_100_data.csv`: S&P 100 con 97 empresas y 2765 días ($lo = 0,005, hi = 0,05$).
- `syp_500_data.csv`: S&P 500 con 457 empresas y 2765 días ($lo = 0,005, hi = 0,02$).

2.5. Clase Portfolio

Para este problema he implementado la clase `Portfolio`, que se encarga de leer los datos de los ficheros en un periodo dado, y generar soluciones aleatorias y arreglarlas, calcular la función fitness para una solución, ver si una solución es válida, etc.

2.6. Representación de la estructura Portfolio

```
1 Estructura Portfolio hereda de Problem<Real>
2   Constantes:
3     tolerancia <- 1e-8
4     lambda <- 500.0
```

```

5  Atributos:
6      retorno : matriz de reales
7      covarianzas : matriz de reales
8      suma_log : vector de reales
9      lo : real
10     hi : real
11     num_fil : entero
12     num_col : entero
13     fecha_inicio : cadena
14     fecha_fin : cadena
15     T : entero
16     pos_inicio : entero
17     pos_fin : entero
18     //Luego ya todos los metodos que ahora explicare

```

Ahora mostramos el constructor de la clase, llamamos a *leerArchivo()*, que leerá los archivos .csv y ajustará *pos-inicio* y *pos-fin*, y *calcularMatrizCovarianzas()* y *calcularSumaLog()* que hacen lo que su propio nombre indica.

```

1  procedimiento Constructor(ruta : cadena, lowest : real, highest : real,
2      fecha_ini : cadena, fecha_final : cadena)
3      pos_inicio <- 0, pos_fin <- 0
4      fecha_inicio <- fecha_ini, fecha_fin <- fecha_final, T <- 0
5      leerArchivo(ruta, retorno)
6      lo <- lowest, hi <- highest
7      num_fil <- tamaño(retorno), num_col <- 0
8      si num_fil > 0 entonces
9          num_col <- tamaño(retorno[0])
10     fin si
11     covarianzas <- matriz(num_col x num_col) inicializada a 0
12     calcularMatrizCovarianzas()
13     suma_log <- vector de tamaño num_col inicializado a 0
14     calcularSumaLog()
15 fin procedimiento

```

2.7. Función Objetivo

Función objetivo a maximizar: equilibra el beneficio obtenido penalizando el riesgo

$$fitness(w) = Beneficio(w) - \lambda \cdot Riesgo(w)$$

Donde usaremos un valor de $\lambda = 500$ En pseudocódigo queda así:

```

1  function fitness(solution : Solucion<Real>) : real
2      retornar beneficio(solution) - lambda * riesgo(solution)
3  fin funcion
4  -----
5  function beneficio(solution : Solucion<Real>) : real
6      tam <- getSolutionSize(), beneficio_total <- 0
7      para i <- 0 hasta tam-1 hacer
8          si solution[i] != 0 entonces
9              beneficio_total <- beneficio_total +
10                  solution[i] * suma_log[i]
11      fin si
12  fin para
13  retornar beneficio_total
14 fin funcion
15 -----
16 function riesgo(solution : Solucion<Real>) : real
17     tam <- getSolutionSize(), riesgo_total <- 0
18     para i <- 0 hasta tam-1 hacer
19         si solution[i] > 0 entonces
20             para j <- 0 hasta tam-1 hacer
21                 si solution[j] > 0 entonces
22                     riesgo_total <- riesgo_total + solution[i]
23                         * covarianzas[i][j] * solution[j]
24             fin si
25     fin para
26     retornar riesgo_total
27 fin funcion

```

```

25         fin para
26     fin si
27 fin para
28 retornar riesgo_total
29 fin funcion

```

2.8. Operadores Comunes

2.8.1. Solución válida

Para ver si una solución es válida debe de cumplir las dos condiciones anteriores.

```

1 funcion isValid(solution : Solucion<Real>) : booleano
2     suma_es_1 <- falso, entre_rangos_o_cero <- verdadero
3     i <- 0, suma <- 0, tam <- getSolutionSize()
4     mientras entre_rangos_o_cero y i < tam hacer
5         valor <- solution[i]
6         entre_rangos_o_cero <- (valor = 0) o (lo <= valor <= hi)
7         suma <- suma + valor
8         i <- i + 1
9     fin mientras
10    si no entre_rangos_o_cero entonces
11        retornar falso
12    fin si
13    suma_es_1 <- igualConTolerancia(suma, 1)
14    retornar suma_es_1 y entre_rangos_o_cero
15 fin funcion

```

2.8.2. Generación de soluciones

Genera una solución aleatoria válida.

```

1 funcion createSolution() : Solucion<Real>
2     num_ceros <- aleatorio(1, num_col - 1)
3     solution <- vector de tamaño num_col
4     para i <- 0 hasta num_col - 1 hacer
5         si i < (num_col - num_ceros) entonces
6             solution[i] <- aleatorio(lo, hi)
7         sino
8             solution[i] <- 0
9         fin si
10    fin para
11    si isValid(solution) entonces
12        barajar(solution)
13        retornar solution
14    fin si
15    fix(solution)
16    retornar solution
17 fin funcion

```

2.8.3. Reparación de soluciones

Para reparar una solución aleatoria no válida, lo que hacemos es primero normalizar para que valga 1 la suma, y después en (1) lo que hacemos es dejar a 0 los que están por debajo de *lo* y en *hi* los que están por encima de *hi*, y los ponemos todos esos valores a la izquierda del vector por comodidad (luego barajamos), y el resto lo llenamos de 0. Nos queda un sobrante para repartir que llamamos suma-acumulada. Luego lo que hacemos es ir despertando los ceros y asignándole un valor si puede entre *lo* y el mínimo entre la suma-acumulada y *hi*, y si es menor que lo, entonces lo repartimos aleatoriamente entre los que no son 0. En el caso de que no queden más ceros, lo que hacemos es repartirlo aleatoriamente entre todos. Por último barajamos para mezclar todo.

```

1 procedimiento fix(solution : referencia Solucion<Real>)
2     suma_pesos <- calculaSuma(solution)
3     normalizar(solution, suma_pesos)
4     tam <- getSolutionSize()
5     // (1) Barrido: mantener valores validos y compactar a la izquierda

```

```

6   num_no_ceros <- 0
7   para i <- 0 hasta tam-1 hacer
8       si solution[i] != 0 y solution[i] >= lo entonces
9           solution[num_no_ceros] <- min(solution[i], hi)
10          num_no_ceros <- num_no_ceros + 1
11      fin si
12  fin para
13  // Rellenar el resto con ceros
14  para i <- num_no_ceros hasta tam-1 hacer
15      solution[i] <- 0
16  fin para
17  suma_acumulada <- 1 - calculaSuma(solution)
18  indice <- 0
19  indice_2 <- aleatorio(0, tam-1)
20  // (2) Repartimos entre los 0
21  mientras suma_acumulada > tolerancia hacer
22      si num_no_ceros < tam entonces
23          si suma_acumulada >= lo entonces
24              valor <- aleatorio(lo, min(hi, suma_acumulada))
25              solution[num_no_ceros] <- valor
26              suma_acumulada <- suma_acumulada - valor
27              num_no_ceros <- num_no_ceros + 1
28          sino
29              valor <- aleatorio(0, solution[indice] - lo)
30              solution[indice] <- solution[indice] - valor
31
32              si suma_acumulada + valor >= lo entonces
33                  solution[num_no_ceros] <- suma_acumulada + valor
34                  suma_acumulada <- 0
35              sino
36                  indice <- (indice + 1) mod num_no_ceros
37                  suma_acumulada <- suma_acumulada + valor
38          fin si
39      fin si
40      sino // Ya no quedan ceros en solution
41          si solution[indice_2] + suma_acumulada <= hi entonces
42              solution[indice_2] <- solution[indice_2] + suma_acumulada
43              suma_acumulada <- 0
44          sino
45              valor <- aleatorio(0, hi - solution[indice_2])
46              solution[indice_2] <- solution[indice_2] + valor
47              suma_acumulada <- suma_acumulada - valor
48              indice_2 <- (indice_2 + 1) mod tam
49          fin si
50      fin si
51  fin mientras
52  barajar(solution)
53 fin procedimiento

```

El procedimiento consta de tres fases:

- Normalización
- Filtrado y compactación
- Redistribución del peso restante

2.8.4. Otros métodos

Método para comparar si dos números reales son iguales:

```

1  funcion igualConTolerancia(num1 : real, num2 : real) : booleano
2      retornar valor absoluto(num1 - num2) <= tolerancia
3  fin funcion

```

Luego también tenemos todos los getters.

3. Descripción de los algoritmos

3.1. Algoritmo Greedy

El algoritmo Greedy que hemos utilizado consiste en ordenar las empresas según una heurística que ahora mostraremos, y la solución se construye de forma que en el orden anterior vamos asignando el máximo peso posible (dentro de las restricciones) a esa empresa hasta no poder asignar a más empresas.

- Función heurística Para el algoritmo greedy utilizaremos la siguiente función heurística:

$$heur(i) = \left(\sum_t \log(1 + \mu_{ti}) \right) - \lambda \cdot \left(\sum_j \Sigma_{ij} \right) / |N|$$

En pseudocódigo:

```
1  para i <- 0 hasta tam_sol - 1 hacer
2      suma_cov <- 0
3      para t <- 0 hasta tam_sol - 1 hacer
4          suma_cov <- suma_cov + getCovarianza(t, i)
5      fin para
6      heuristica[i] <- getSumaLog(i) - lambda * (suma_cov / tam_sol)
7  fin para
```

- Esquema general

```
1  funcion optimize(problem, maxevals) : Resultado
2      Obtenemos heurísticas //Como se hace en el apartado anterior
3      tam_sol <- getSolutionSize()
4      indices <- [0, 1, ..., tam_sol-1]
5      ordenar indices de forma descendente segun heuristica
6      sol <- vector de tamaño tam_sol
7      a_repartir <- 1.0
8      hi <- getHi()
9      para i <- 0 hasta tam_sol - 1 hacer
10         si a_repartir > 0 entonces
11             asignacion <- min(hi, a_repartir)
12             sol[indices[i]] <- asignacion
13             a_repartir <- a_repartir - asignacion
14         sino
15             sol[indices[i]] <- 0
16         fin si
17     fin para
18     retornar (sol, fitness(sol), 1)
19 fin funcion
```

3.2. Búsqueda Local (BL)

El algoritmo de búsqueda local consiste en primero crear una solución inicial aleatoria, se aplica una búsqueda local tipo primero mejor, explorando un vecindario formado por todos los pares de posiciones $(i, j) \forall i, j \in 0, 1, \dots, N$, donde nos movemos a la primera solución mejor. Para cada par, se genera un vecino transfiriendo un porcentaje del peso desde la posición i a la posición j . Este porcentaje es el siguiente:

En la posición i el nuevo valor pasa a ser $(1 - ratio) \cdot w_i$, y en la posición j el nuevo valor pasa a ser $w_j + ratio \cdot w_i$. El máximo valor que tomará la variable será $ratio = 0,4$

```
1  funcion optimize(problem, maxevals) : Resultado
2      // (1) Generacion de la solucion inicial:
3      sol <- createSolution()
4      tam <- getSolutionSize()
5
6      // (2) Exploracion del entorno:
7      posiciones <- lista de todos los pares (i, j) con i, j en [0, tam-1]
8      barajar(posiciones)
9
10     pos <- 0, evals <- 0
11     fit <- fitness(sol)
```

```

12 (lo, hi) <- getSolutionDomainRange()
13 ratio_max <- 0.4
14
15 mientras pos < tam*tam y evals < maxevals hacer
16   (i, j) <- posiciones[pos]
17   si i != j entonces
18     val_i <- sol[i]
19     val_j <- sol[j]
20     si val_i > lo y val_j < hi entonces
21       tope_1 <- 1 - (lo / val_i)
22       tope_2 <- (hi - val_j) / val_i
23       tope_3 <- ratio_max
24       ratio <- min(tope_1, tope_2, tope_3)
25       si ratio = 0 entonces
26         pos <- pos + 1
27         continuar
28       fin si
29
30       // (3) Generacion de vecino:
31       sol[i] <- (1 - ratio) * val_i
32       sol[j] <- val_j + ratio * val_i
33       si isValid(sol) entonces // Es necesario porque a veces por redondeo
34         // puede poener un valor por debajo de lo
35         new_fit <- fitness(sol)
36         evals <- evals + 1
37
38         // (4) Criterio de aceptacion:
39         si new_fit > fit entonces
40
41           //(5) Reinicio de la busqueda
42           barajar(posiciones)
43           pos <- 0
44           fit <- new_fit
45         sino
46           sol[i] <- val_i
47           sol[j] <- val_j
48           pos <- pos + 1
49         fin si
50       sino
51         sol[i] <- val_i
52         sol[j] <- val_j
53         pos <- pos + 1
54       fin si
55     sino
56       pos <- pos + 1
57     fin si
58   sino
59     pos <- pos + 1
60   fin mientras
61   retornar (sol, fit, evals)
62 fin funcion

```

- (1) Generación de la solución inicial: Se obtiene una solución aleatoria mediante createSolution().
- (2) Exploración del entorno: Se generan todos los pares (i, j) y se recorren en orden aleatorio.
- (3) Generación de vecino: Se transfiere una proporción del peso desde la posición i a la j.
- (4) Criterio de aceptación: Se acepta el nuevo vecino si mejora el valor de la función objetivo.
- (5) Reinicio de la búsqueda: Cuando se encuentra una mejora, se vuelve a explorar el vecindario desde el inicio.

4. Estructura del código y manual de usuario

4.1. Estructura del código

Esta es la estructura general del proyecto, donde en la carpeta build están los archivos generados automáticamente durante la compilación del proyecto, en configuraciones están las configuraciones de cada algoritmo, en datos-portfolio-2526 los csv con los datos de las bolsas, en inc los archivos .h y en src los .cpp, en common los archivos comunes a todos, y en resultados las gráficas y el script para generarlas.

```
|-- build
|-- CMakeLists.txt
|-- common
|-- configuraciones
|-- datos_portfolio_2526
|-- ejecutar_todo.sh
|-- inc
|-- resultados
|-- LICENSE
|-- main.cpp
|-- README.md
|-- src
```

4.2. Compilación

Nos situamos en la carpeta software y hacemos lo siguiente:

- cd build
- rm -rf *
- cmake ..
- make
- cd ..

4.3. Ejecución

Ahora después de hacer la compilación anterior nos situamos en la carpeta software y hacemos:

- ./ejecutar_todo.sh semilla_inicial
- Ejemplo de uso ./ejecutar_todo.sh 3 empezará con la semilla 3 y terminará con 53 la última ejecución

4.4. Gráficas

Nos situamos en la carpeta software de nuevo:

- cd resultados
- python3 graficas.py

5. Experimentos y análisis de resultados

5.1. Descripción de los experimentos

Se utilizan datos bursátiles reales (2015-2025) de empresas con presencia continua en las siguientes bolsas de valores:

- `ibex_35_data.csv`: IBEX 35 con 30 empresas y 2813 días. Parámetros ($lo = 0,005, hi = 0,08$).
- `syp_100_data.csv`: S&P 100 con 97 empresas y 2765 días. Parámetros ($lo = 0,005, hi = 0,05$).
- `syp_500_data.csv`: S&P 500 con 457 empresas y 2765 días. Parámetros ($lo = 0,005, hi = 0,02$).

5.1.1. Algoritmos comparados

Los algoritmos serán:

- Aleatorio (RS)
- Búsqueda Local (BL)
- Greedy (GR)

Para que la comparación sea justa se utilizará un máximo de 10000 evaluaciones, una evaluación es cada vez que llamamos a la función *fitness*

- $maxevals = 10000$

Se va a utilizar como semilla desde 1, sumando 1 en cada iteración hasta llegar a 50 incluido.

5.2. Resultados

Resultados medios de los tres algoritmos en cada bolsa:

Algoritmo	Fitness (2015–2024)	Fitness (2025)	Beneficio (2025)	Evaluaciones	Tiempo (s)
Greedy	-0.2128	0.0907	0.1152	1.0000	0.0287
Random	-0.7913	0.0806	0.1069	10000.0000	0.1743
BL	-0.3142	0.0981	0.1225	1306.8600	0.0265

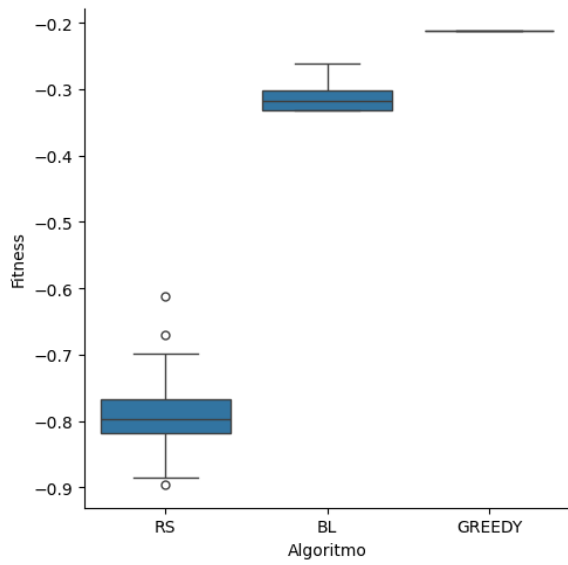
Cuadro 2: Ibex35 resultados (medias)

Algoritmo	Fitness (2015–2024)	Fitness (2025)	Beneficio (2025)	Evaluaciones	Tiempo (s)
Greedy	0.8666	-0.0407	0.0099	1.0000	0.0520
Random	0.3754	0.0107	0.0557	10000.0000	0.7621
BL	0.6844	-0.0182	0.0275	6010.4400	0.7446

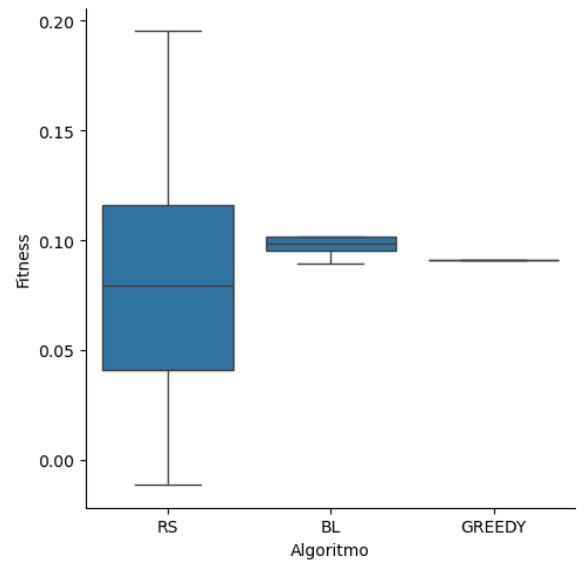
Cuadro 3: SyP100 resultados (medias)

Algoritmo	Fitness (2015–2024)	Fitness (2025)	Beneficio (2025)	Evaluaciones	Tiempo (s)
Greedy	1.0430	-0.1157	-0.0628	1.0000	0.3512
Random	0.1814	-0.0363	0.0095	10000.0000	4.0448
BL	0.4767	-0.0620	-0.0096	10000.0000	13.3834

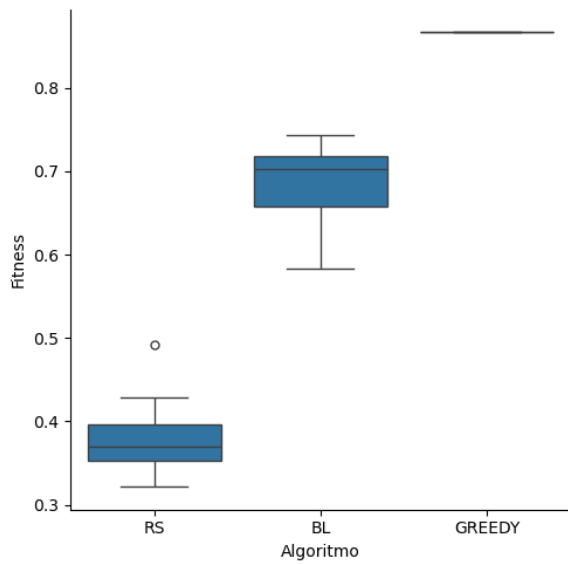
Cuadro 4: SyP500 resultados (medias)



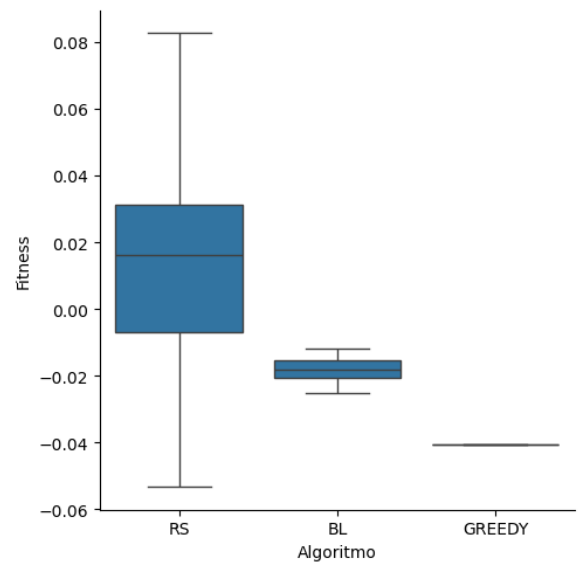
(a) IBEX35 2015-2024



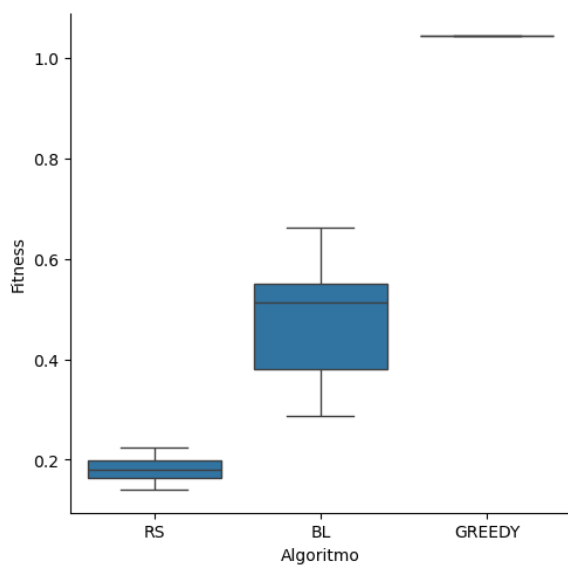
(b) IBEX35 2025



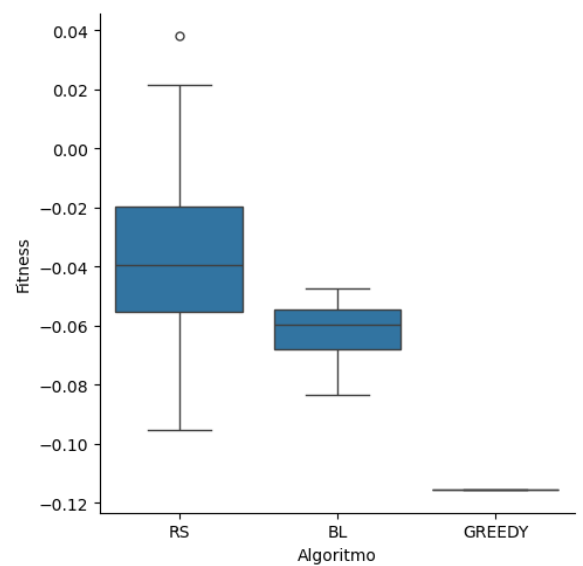
(c) SyP100 2015-2024



(d) SyP100 2025



(e) SyP500 2015-2024



(f) SyP500 2025

Figura 1: Conjunto de resultados

5.3. Análisis de resultados

5.3.1. Aleatorio

El algoritmo Aleatorio genera soluciones de forma aleatoria, por lo que no depende de ninguna heurística ni de la estructura del problema. Esto implica una alta variabilidad en los resultados, pero también en la posibilidad de encontrar soluciones que pueden "por suerte" llegar a ser buenas.

- 2014-2024 Como era de esperar al ser aleatorio presenta un rendimiento inestable, algunas soluciones son buenas y otras no, por eso en los boxplots se puede observar que sus cajas son grandes. Además como se muestra en los boxplots tiene algunos puntos fuera de la caja que son valores atípicos, algo que los otros algoritmos no presentan. Tienen sentido los resultados del *fitness* en este periodo, en comparación con los otros algoritmos, este siempre toma soluciones aleatorias y nunca las mejora como en Búsqueda Local y no tiene la información que maneja Greedy, por eso se supone que debería de ser el peor. Los resultados así lo confirman, teniendo el peor *fitness* en el periodo histórico en las tres bolsas.
- 2025 Según los resultados obtenidos en 2014-2024, el algoritmo debería de ser el peor en el *fitness* de 2025 también, pero eso solo sucede en el IBEX35, y de hecho en el SYP100 y SYP500 tiene el mejor *fitness*. Esto sucede porque a pesar de ser el peor en el anterior, las empresas del IBEX35 están mas diversificadas que en el SYP100 y SYP500, donde muchas de ellas son grandes tecnológicas, por lo que en el periodo histórico de 2014-2024 estas empresas gracias a la pandemia tenían retornos mucho mayores, luego en el histórico salían ganando. Sin embargo la gran mayoría de estas en 2025 no siguen el mismo ritmo y han bajado mucho. Por esta razón los algoritmos de Búsqueda Local y Greedy, tomaban más empresas tecnológicas que el Aleatorio, y cuando hacemos el test con las de 2025 el Aleatorio sale ganando al tener menos tecnológicas y diversificar más. Y por esto en el IBEX35 que tiene empresas de más sectores, donde apenas hay tecnológicas, no se ve afectado por la pandemia y tiene sentido que siga siendo el peor.

5.3.2. Búsqueda Local

El algoritmo Búsqueda Local genera una solución aleatoria y a partir de ella se va moviendo al primer mejor vecino.

- 2014-2024 Presenta una gran mejoría respecto al Aleatorio, ya que modifica la solución a mejores, en el IBEX35 al tener menos empresas el número de vecinos, y por tanto de "mejoras" posibles a realizarle es menor, por eso nunca llega a las 10000 evaluaciones y siempre sale antes. En el SYP100 ya casi que llega al máximo número de evaluaciones y en el SP500 siempre sale antes, algo que tiene sentido porque en el SYP 500 tenemos un vector de vecinos de tamaño alrededor de $500 \cdot 500 = 250000$ que es mayor a 10000 por lo que es imposible que lo recorra entero. En los boxplots también las cajas son grandes, porque al depender desde el principio de una solución aleatoria hace que haya una gran variedad al final, porque hay algunas soluciones que si son muy malas desde el principio tampoco se pueden llegar a mejorar mucho. De hecho por esta razón cuantas menos empresas hay más se acerca el *fitness* al mejor, que es el Greedy, ya que permite realizar cuantas menos empresas más cambios y explorar más vecinos y mejores soluciones, y cuantas más empresas menos cambios podemos realizar (como comentaba antes en el ibex hay 30 empresas por lo que en el vector de vecinos hay $30 \times 30 = 900$, se puede explorar muchas mas veces y mejorar que en el SyP500 que hay $500 \times 500 = 250000$) por esto cuantas más empresas más lejos del Greedy está.
- 2025 Sucede lo que he explicado anteriormente de las empresas tecnológicas, por eso en SyP100 y SyP500 es mejor que el Greedy y peor que el Aleatorio. En el IBEX35 es el mejor por muy poco, es similar al Greedy pero igual más o menos que en el periodo histórico por lo que como tal no nos dice nada.

5.3.3. Greedy

Es el único algoritmo determinista, por eso solo lo ejecutamos una vez, y es el que dispone de más información para hacer su heurística

- 2014-2024 Lo primero los boxplots son una sola línea ya que solo se ejecuta una vez. En el periodo histórico es el mejor de los tres algoritmos en todos los casos, ya que este dispone de más información al tener la heurística que los otros, y que tenga el mejor *fitness* se debe a que la heurística es muy similar a la propia función *fitness*, por tanto va dando más peso a empresas que luego al ser tan parecida tendremos un buen *fitness*.
- 2025 Lo primero los boxplots son una sola línea ya que solo se ejecuta una vez. En 2025 la cosa cambia mucho por lo explicado anteriormente sobre las empresas tecnológicas, por eso es el peor en el SyP100 y SyP500 porque seguramente tomará muchas empresas tecnologicas algo que no sucede en el IBEX35 ya que casi que no hay.

6. Referencias

1. Guión de las prácticas y material de teoría de la asignatura.
2. OpenAI. (2026). *ChatGPT* (Versión GPT-4o) <https://chatgpt.com>
3. Google. (2026). *Gemini* (Versión 1.5 Pro) <https://gemini.google.com>